

ASSESSMENT AND INTERNAL VERIFICATION FRONT SHEET (Individual Criteria)

(Note: This version is to be used for an assignment brief issued to students via Classter)

Course Title	IT6-A04-23 Bachelor of Science (Honours) in Digital Games Development			Lecturer Name & Surname	David Deguara	
Unit Number & Title		ITDGM-606-2312 Multiplayer Game Development				
Assignment Number, Title / Type		A02: Developing a Server Authoritative Multiplayer Game				
Date Set		16 th December 2025	Deadline Date	19 th January 2026		
Student Name			ID Number		Class / Group	

Assessment Criteria	Maximum Mark
A&A1: Analyse the requirements for implementing matchmaking and lobby systems.	7
A&A2: Apply authoritative server techniques to prevent cheating in multiplayer games.	7
A&A3: Implement networked player movement synchronisation.	7
E&C2: Create a working multiplayer game prototype demonstrating core gameplay and network features.	10
R&U7: Describe data mining applications in analysing player behaviour.	5
A&A4: Apply optimisation techniques to reduce network traffic and improve performance.	7
A&A5: Develop a plan to integrate data mining insights for improving multiplayer game design.	7
E&C3: Propose a scalable solutions for multiplayer game servers.	10
Total Mark	60

Notes to Students:

- This assignment brief has been approved and released by the Internal Verifier through Classter.
- Assessment marks and feedback by the lecturer will be available online via Classter (<http://mcast.classter.com>) following release by the Internal Verifier
- Students submitting their assignment on Moodle/Turnitin will be requested to confirm online the following statements:

Student's declaration prior to handing-in of assignment

- ❖ I certify that the work submitted for this assignment is my own and that I have read and understood the respective Plagiarism Policy

Student's declaration on assessment special arrangements

- ❖ I certify that adequate support was given to me during the assignment through the Institute and/or the Inclusive Education Unit.
- ❖ I declare that I refused the special support offered by the Institute.

A02: Developing a Server Authoritative Multiplayer Game

1. Important Instructions

- **Plagiarism:** All work must be original. Plagiarism or use of unauthorised materials will result in disciplinary action.
- **Individual Assessment:** While Section 1 is a group project, individual contributions will be assessed through assigned functionalities, the Q&A session, and version control history. Section 2 is entirely individual.
- **Late Submissions:** Late submissions will be penalised according to MCAST regulations.
- **Technical Issues:** Ensure your submission is complete and all files can be opened.

2. Introduction

This assignment is divided into two sections:

- **Section 1 (Teamwork - 43%):** Work collaboratively in groups of 2-3 students to design, develop, and deploy a server authoritative multiplayer game using Unity 6 and Unity Gaming Services. Each student has individual responsibilities within the team project.
- **Section 2 (Individual - 17%):** Complete an advanced individual task demonstrating independent mastery of multiplayer networking concepts.

The team game must be a 3D, turn-based, co-operative (co-op) multiplayer experience. All content must be original and created from scratch.

3. Assessment Overview

Section	Type	Weighting
Section 1: Multiplayer Game	Teamwork (with individual components)	43%
Section 2: Advanced Individual Task	Individual	17%
Total		60%

Section 1 – Teamwork (43%)

4. Task Description

4.1 Overview

Develop a server authoritative multiplayer game using Unity 6 with the following Unity Gaming Services:

- Unity Netcode for GameObjects
- Multiplayer Lobbies
- Authentication
- Relay
- Unity Analytics

4.2 Game Requirements

Mandatory Requirements:

1. **3D Game:** The game must be developed in 3D using Unity 6.
2. **Turn-Based Gameplay:** The game must implement turn-based mechanics where players take actions in sequence.
3. **Co-operative (Co-op):** Players must work together towards a common goal.
4. **Server Authoritative Architecture:** The server (host) must validate all game actions to prevent cheating.
5. **Original Content:** All content must be created from scratch.

Technical Requirements:

1. **Matchmaking and Lobby System:** Implement a functional lobby system using Unity Lobby service.
2. **Player Synchronisation:** Implement networked player movement and game state synchronisation.
3. **Authentication:** Implement Unity Authentication for player identity management.
4. **Relay Service:** Use Unity Relay for peer-to-peer connectivity.
5. **Analytics Integration:** Implement Unity Analytics to collect player behaviour data.

4.3 Individual Responsibilities within the Team

IMPORTANT: Although this is a team project, each student must have clearly defined individual responsibilities. This ensures fair assessment even if team dynamics are unequal.

Proposal Requirement:

Before development begins, your team must submit a proposal that includes:

- Game title and short description of the game concept
- **Clear list of multiplayer functionalities assigned to each student (minimum two per student).** Each functionality must involve code implementation.
- Brief description of how the game will incorporate server authoritative logic

ITDGM-606-2312: Multiplayer Game Development

The proposal must be submitted and approved by your lecturer before development can begin.

Each student must:

1. Develop at least **two assigned multiplayer functionalities** (these will be graded individually)
2. Contribute to the overall game design and environment
3. Participate actively in team integration, testing, and refinement
4. Compile an individual progress report (journal) describing weekly contributions

Example Multiplayer Functionalities (for proposal guidance):

Functionality Category	Examples
Lobby & Matchmaking	Lobby creation, lobby browser, ready system, player slots
Player Actions	Turn-based action system, player abilities, movement sync
Game State Management	Turn manager, win/lose conditions, score synchronisation
UI & Feedback	Networked chat, player indicators, turn notifications
Server Authority	Action validation, anti-cheat checks, server-side game logic

4.4 Teamwork Regulations

- If a student is not cooperating with their team members, the lecturer has the right to remove that student from the team and assign them to complete the project individually, with a significant mark deduction.
- If a student is going to miss a lecture during assigned sessions, they must inform their lecturer and their team by 9 AM on that day.
- Students must meet outside lecture hours to collaborate on the development. A minimum of 5 documented meetings is required.

4.5 Deliverables

Your group must submit the following:

1. **Unity Project:** Complete Unity 6 project folder with all source files.
2. **Windows Build:** A working Windows executable build of the game.
3. **Gameplay Video:** A video demonstration (5-10 minutes) showing the multiplayer game being played.
4. **Technical Documentation:** A written report (2000-3000 words) covering network architecture, matchmaking analysis, anti-cheat strategies, data mining applications, optimisation techniques, and scalability proposals.
5. **Individual Progress Reports:** Each student must submit their own progress journal. Compile into a single shared document, clearly separated by student name.
6. **Git/Plastic SCM Repository History:** Export of complete version control history.
7. **Team Contribution Declaration:** Signed declaration form detailing each member's assigned functionalities and meeting log.

5. Version Control Requirements

All groups are **required** to use version control throughout the development process:

- **Tool:** Use either Git or Plastic SCM for version control.
- **Starting Point:** You must start from scratch with a new repository.
- **Commit History:** Regular commits with meaningful messages are expected.
- **Submission:** Include the complete repository history as part of your submission.

6. Presentation and Q&A

Following the submission deadline, all groups will present their work to the class:

1. Demonstrate the working multiplayer game
2. Each student explains their assigned functionalities and implementation
3. Discuss challenges faced and solutions implemented
4. Answer questions from the lecturer and peers

The Q&A session will verify individual understanding. **Each student must be prepared to answer questions specifically about their assigned functionalities.**

7. Section 1 Assessment Criteria

Team Assessment (Shared Marks):

Code	Task Description	Marks
A&A1	Analyse the requirements for implementing matchmaking and lobby systems.	7%
E&C2	Create a working multiplayer game prototype demonstrating core gameplay and network features.	10%
Team Subtotal		17%

Individual Assessment (Per Student):

Code	Task Description	Marks
A&A2	Apply authoritative server techniques to prevent cheating in multiplayer games.	7%
A&A3	Implement networked player movement synchronisation.	7%
R&U7	Describe data mining applications in analysing player behaviour.	5%
A&A5	Develop a plan to integrate data mining insights for improving multiplayer game design.	7%
Individual Subtotal		26%
Section 1 Total		43%

Note: A&A2 and A&A3 are assessed through your individually assigned functionalities as specified in your approved proposal. Team members share the 17% from A&A1 and E&C2 equally.

Section 2 – Individual (17%)

8. Advanced Individual Task

This section assesses your individual mastery of advanced multiplayer networking concepts. This task must be completed **entirely on your own** without collaboration with team members.

8.1 Task: Scalable Multiplayer Server Solution

You are required to propose and implement a scalable and optimised solution for multiplayer games in the form of a Network State Recording and **Replay System**.

Implementation direction:

- Create a new Unity project using Netcode for GameObjects (you may use grey-box assets)
- Implement a **Record** button that begins logging network state data to a file (JSON, CSV, or binary format)
- Implement a **Replay** button that loads saved data and reproduces the recorded session accurately

If implementing a recorder, the recording should capture:

- Player positions and rotations (all connected clients)
- NetworkVariable state changes
- RPC calls and their parameters
- Object spawning and despawning events
- Timestamps for all events

Key Requirements:

- Your solution must demonstrate understanding of scalability challenges in multiplayer games
- Include a written explanation of how your solution addresses scalability
- Provide a working demonstration with clear documentation

8.2 Deliverables

- **Unity Project:** Submit the complete Unity project (without the Library folder)
- **Video Demonstration:** A screen-recorded video (6-8 minutes) showing your scalable solution in action, with a clear explanation of the key scripts and logic used.
- **Written Proposal:** A document (800-1200 words) explaining your scalable solution, its architecture, and how it could be adapted for larger multiplayer environments.

8.3 Section 2 Assessment Criteria

Code	Task Description	Marks
E&C3	Propose a scalable solution for multiplayer game servers.	10%
A&A4	Apply optimisation techniques to reduce network traffic and improve performance.	7%
Section 2 Total		17%

9. Submission Guidelines

9.1 Section 1 Submission (Team)

File Naming Convention: A02_Section1_GroupNumber_MultiplayerGame.zip

Contents:

- Unity Project folder (zipped)
- Windows Build folder (zipped)
- Gameplay video (MP4 format, maximum 500MB)
- Technical documentation (PDF format)
- Individual progress reports (single PDF, separated by student name)
- Version control history export
- Signed Team Contribution Declaration form

9.2 Section 2 Submission (Individual)

File Naming Convention: A02_Section2_StudentID_ScalableSolution.zip

Contents:

- Unity Project folder without Library folder (zipped)
- Video demonstration (MP4 format, 6-8 minutes)
- Written proposal document (PDF format)

10. Marking Rubrics

Section 1 – Team Assessment

A&A1: Analyse the requirements for implementing matchmaking and lobby systems (7%)

Excellent (7%)	Good (5-6%)	Satisfactory (3-4%)	Needs Work (1-2%)	Not Met (0%)
Comprehensive analysis with detailed justification. Fully functional lobby with advanced features.	Good analysis with clear requirements. Functional lobby with core features.	Basic analysis. Lobby works but lacks some features.	Limited analysis. Lobby partially functional.	No analysis or lobby not implemented.

E&C2: Create a working multiplayer game prototype demonstrating core gameplay and network features. (10%)

Excellent (9-10%)	Good (7-8%)	Satisfactory (5-6%)	Needs Work (2-4%)	Not Met (0-1%)
Polished, complete game. Engaging gameplay. Bug-free multiplayer.	Complete game with most features working. Minor issues.	Functional prototype meeting basic requirements.	Incomplete prototype with missing features.	No working prototype or no multiplayer.

Section 1 – Individual Assessment

A&A2: Apply authoritative server techniques to prevent cheating in multiplayer games (7%)

Excellent (7%)	Good (5-6%)	Satisfactory (3-4%)	Needs Work (1-2%)	Not Met (0%)
Robust server authority implementation. All critical actions validated server-side. Well documented.	Good server authority with most actions validated. Clear documentation.	Basic server validation. Some client-side vulnerabilities remain.	Limited server authority. Significant security gaps.	No server authority implemented.

A&A3: Implement networked player movement synchronisation (7%)

Excellent (7%)	Good (5-6%)	Satisfactory (3-4%)	Needs Work (1-2%)	Not Met (0%)
Smooth, responsive synchronisation. Handles latency gracefully. Interpolation/prediction implemented.	Good synchronisation with minor latency issues. Mostly smooth gameplay.	Basic synchronisation working. Noticeable lag or jitter.	Synchronisation partially working. Significant issues.	No synchronisation implemented.

R&U7: Describe data mining applications in analysing player behaviour (5%)

Excellent (5%)	Good (4%)	Satisfactory (3%)	Needs Work (1-2%)	Not Met (0%)
Comprehensive description with specific examples. Analytics properly implemented.	Good description with relevant examples.	Basic description. Analytics implemented but limited.	Limited description. Analytics partially implemented.	No description or analytics not implemented.

A&A5: Develop a plan to integrate data mining insights for improving multiplayer game design. (7%)

Excellent (7%)	Good (5-6%)	Satisfactory (3-4%)	Needs Work (1-2%)	Not Met (0%)
Detailed plan with specific metrics and actionable improvements. Evidence of data-driven decisions.	Good plan with clear metrics and proposed improvements.	Basic plan with some metrics identified.	Vague plan with few specifics.	No integration plan provided.

Section 2 – Individual Advanced Task

E&C3: Propose a scalable solution for multiplayer game servers (10%)

Excellent (9-10%)	Good (7-8%)	Satisfactory (5-6%)	Needs Work (2-4%)	Not Met (0-1%)
Innovative, well-implemented scalable solution. Comprehensive proposal with clear architecture. Excellent demonstration.	Good scalable solution with clear documentation. Working demonstration with minor issues.	Basic scalable solution. Some features missing. Adequate explanation.	Attempts scalability but lacks functionality. Limited explanation.	Missing or irrelevant work.

A&A4: Apply optimisation techniques to reduce network traffic and improve performance (7%)

Excellent (7%)	Good (5-6%)	Satisfactory (3-4%)	Needs Work (1-2%)	Not Met (0%)
Multiple techniques applied. Efficient serialisation, delta compression. Documented improvements.	Several techniques with measurable improvements.	Basic optimisation. Some awareness but limited implementation.	Minimal optimisation.	No optimisation techniques applied.

Team Contribution Declaration Form

(To be completed and submitted with the Section 1 assignment)

Group Number	
---------------------	--

Team Members and Assigned Functionalities

Full Name	Student ID	Functionality 1	Functionality 2

Meeting Log (Minimum 5 Meetings Required)

No.	Date	Duration	Topics Discussed	Action Items
1				
2				
3				
4				
5				
6				
7				

Signatures

Full Name	Signature	Date

Declaration

We, the undersigned, declare that:

- This work is entirely our own and has been completed from scratch for this assignment.
- We have not reused any work from past or current assignments.
- The assigned functionalities listed above accurately reflect each team member's individual responsibilities.
- We have used version control (Git/Plastic SCM) throughout the development process.
- We understand that plagiarism or misrepresentation of contributions is a serious academic offence.